

INFORME DE GUÍA PRÁCTICA

PORTADA

Tema: Desarrollo de un Aplicativo Interactivo de Lógica, Registro de Calificaciones y Persistencia de Datos en Java

Unidad de Organización Curricular: Básica

Nivel y Paralelo: Primer Nivel - Paralelo A

Alumnos participantes: Isaac Adrián Sanchez Lema

Asignatura: Algoritmos y Lógica de Programación

Docente: Ing. Jose Caiza, Mg.

1. Objetivos

1.1. Objetivo General

Diseñar e implementar un aplicativo interactivo en el lenguaje de programación Java que integre operaciones aritméticas básicas, procesamiento de calificaciones por medio de arreglos unidimensionales y persistencia de datos en archivos locales, garantizando la robustez del sistema a través de métodos de validación cíclicos.

1.2. Objetivos Específicos

- Desarrollar un menú de navegación continuo y modular que controle el estado de los datos procesados, impidiendo la finalización accidental del programa si existen cálculos pendientes de almacenar.
- Aplicar arreglos unidimensionales para la captura y el análisis descriptivo de un conjunto finito de notas, determinando promedios, valores extremos (máximos y mínimos) y estados de aprobación académica bajo una constante de corte fija.
- Implementar un módulo de persistencia en disco que almacene los resultados históricos de las operaciones de forma acumulativa en archivos de texto plano, estructurando la salida con metadatos del usuario y marcas de tiempo del sistema.

2. Modalidad

Presencial / Práctica de Laboratorio Asistida.

3. Tiempo de Duración

- **Presenciales:** 2 horas
- **No presenciales:** 4 hours

4. Instrucciones

- ---

Inicializar el entorno de desarrollo integrado (IDE) compatible con Java SE (JDK 8 o superior).
- Diseñar la arquitectura del programa dividiendo las responsabilidades en métodos estáticos para facilitar la validación y legibilidad del código.
- Asegurar que todas las lecturas de consola limpien los buffers y manejen las excepciones de formato numérico (NumberFormatException).
- Validar que el archivo de texto plano no se sobrescriba en cada ejecución, utilizando el modo apéndice (append).

5. Listado de Equipos, Materiales y Recursos

5.1. Equipos y Materiales Generales

- Computador portátil HP Victus.
- Sistema Operativo Windows 11 / Distribución Linux.
- Kit de desarrollo Java (JDK 17).
- Entorno de Desarrollo (VS Code / IntelliJ IDEA / NetBeans).

5.2. TAC (Tecnologías para el Aprendizaje y Conocimiento)

- [x] Plataformas educativas (Formatos institucionales)
- [x] Inteligencia Artificial (Asistencia en análisis lógico)

6. Análisis del Problema

6.1. Configuración Inicial e Interfaz de Usuario

Entrada: Nombre del estudiante (cadena de texto alfabética) y lenguaje de programación (cadena de texto).

Proceso: Validar mediante expresiones regulares que el nombre contenga únicamente caracteres alfabéticos y espacios. Forzar la captura antes de habilitar el menú principal.

Salida: Interfaz visual en consola que muestra el perfil del estudiante activo y las opciones del sistema.

6.2. Módulo de Operaciones Matemáticas

Entrada: Dos números reales de doble precisión (num1 y num2).

Proceso: Calcular la suma, resta y multiplicación. Para la división, evaluar mediante una estructura condicional si el divisor (num2) es diferente de cero; si es cero, mitigar el error devolviendo una cadena de advertencia controlada.

Salida: Despliegue de resultados formateados a dos decimales y transferencia de datos al búfer dinámico.

6.3. Módulo de Registro de Notas

Entrada: Cinco calificaciones numéricas consecutivas.

Proceso: Validar que cada nota se encuentre en el rango cerrado [0.0, 10.0]. Almacenar los datos en un vector. Inicializar las variables de extremos con el primer elemento del arreglo (notas[0]). Recorrer el vector en un ciclo iterativo para calcular la sumatoria, evaluar los valores máximos y mínimos, y contabilizar los aprobados o reprobados en función de la constante límite (7.0).

Salida: Reporte estadístico en pantalla y adición del bloque de texto al acumulador del sistema.

6.4. Módulo de Persistencia Histórica

Entrada: Nombre del estudiante, lenguaje activo, bloque de resultados acumulados en el objeto StringBuilder y fecha/hora del sistema.

Proceso: Verificar la existencia de la carpeta destino; si no existe, crearla dinámicamente. Instanciar un canal de escritura física en modo apéndice para añadir los datos al final del archivo resultados.txt. Modificar el estado de la bandera de control (datosProcesados = false) y limpiar el búfer de memoria.

Salida: Archivo físico actualizado en disco y mensaje de éxito en consola.

7. Tabla de Variables del Sistema

Identificador	Tipo de Dato	Alcance	Descripción / Propósito Lógico
NOTA_MINIMA_APROBACION	double (Constante)	Global de clase	Umbral académico fijo para determinar la aprobación (7.0).
scanner	Scanner (Constante)	Global de clase	Instancia única para la captura de flujos de entrada por teclado.
opcion	int	Local (main)	Almacena la selección del menú seleccionada por el usuario.
nombreEstudiante	String	Local (main)	Registra la identidad del alumno operador del sistema.
lenguajeUtilizado	String	Local (main)	Registra el entorno o lenguaje de desarrollo

Identificador	Tipo de Dato	Alcance	Descripción / Propósito Lógico
			configurado.
resultadosAcumulados	StringBuilder	Local (main)	Búfer dinámico que colecta los reportes de los módulos en memoria.
datosProcesados	boolean	Local (main)	Bandera (flag) de control de estado para cambios pendientes sin guardar.
num1, num2	double	Local (ejecutarOperacionesBasicas)	Operandos de entrada para el procesamiento aritmético.
suma, resta, multiplicacion	double	Local (ejecutarOperacionesBasicas)	Almacenan los resultados de las operaciones aritméticas directas.
divisionStr	String	Local (ejecutarOperacionesBasicas)	Maneja el resultado de la división o el mensaje de error controlado.
notas	double[]	Local (ejecutarRegistroNotas)	Arreglo unidimensional (vector) de tamaño fijo 5 para las calificaciones.
sumaNotas	double	Local (ejecutarRegistroNotas)	Acumulador para el cálculo de la media aritmética.
notaMayor,	double	Local (ejecutarRegistroNotas)	Variables de control estadístico para

Identificador	Tipo de Dato	Alcance	Descripción / Propósito Lógico
notaMenor		as)	almacenar los valores extremos.
aprobados, reprobados	int	Local (ejecutarRegistroNotas)	Contadores para la clasificación del estado académico estudiantil.
carpetaDestino, rutaArchivo	String	Local (guardarResultados)	Cadenas de texto que definen la ubicación física del archivo en disco.

8. Algoritmo Lógico Basado en Pasos

1. _____
- Inicio del Programa.**
2. **Fase de Inicialización del Perfil:**
 - 2.1. Solicitar el ingreso del nombre del estudiante.
 - 2.2. Si la entrada está vacía o contiene números/símbolos, mostrar error y repetir el paso 2.1.
 - 2.3. Solicitar el ingreso del lenguaje de programación. Si está vacío, repetir el paso 2.3.
3. **Despliegue del Menú Continuo (Bucle principal):**
 - 3.1. Imprimir la interfaz en consola con los datos del perfil y las opciones [1 - 4].
 - 3.2. Leer la variable opcion asegurando que sea un entero válido entre 1 y 4.
4. **Evaluación de la Opción Seleccionada (switch):**
 - **Si opcion == 1 (Operaciones Básicas):**
 - 4.1.1. Leer num1 y num2 de forma segura (reemplazar comas por puntos si existen).
 - 4.1.2. Calcular suma, resta y multiplicación.
 - 4.1.3. Si num2 != 0, calcular división y formatear; de lo contrario, definir mensaje de error para división por cero.
 - 4.1.4. Imprimir resultados, estructurar la cadena de texto y anexarla a resultadosAcumulados.
 - 4.1.5. Asignar datosProcesados = true. Volver al paso 3.
 - **Si opcion == 2 (Registro de Notas):**
 - 4.2.1. Iniciar un ciclo desde i = 0 hasta 4.
 - 4.2.2. Leer la nota de la asignatura i+1, validando que se encuentre entre 0.0 y 10.0.
 - 4.2.3. Asignar notaMayor = notas[0] y notaMenor = notas[0].
 - 4.2.4. Recorrer el arreglo: sumar cada nota a sumaNotas, actualizar notaMayor y notaMenor según corresponda, e incrementar los contadores de aprobados o reprobados comparando contra la constante 7.0.
 - 4.2.5. Calcular el promedio general (sumaNotas / 5).
 - 4.2.6. Imprimir métricas, estructurar el reporte y anexarlo a resultadosAcumulados.

- 4.2.7. Asignar `datosProcesados = true`. Volver al paso 3.
 - **Si opcion == 3 (Guardar Resultados):**
 - 4.3.1. Si `datosProcesados == true`, verificar/crear la carpeta "resultados", abrir el archivo en modo apéndice, escribir los metadatos junto con la cadena de `resultadosAcumulados`, limpiar el búfer, cambiar `datosProcesados = false` y volver al paso 3.
 - 4.3.2. Si `datosProcesados == false`, mostrar mensaje de advertencia y volver al paso 3.
 - **Si opcion == 4 (Salir):**
 - 4.4.1. Si `datosProcesados == true`, advertir sobre cálculos pendientes sin guardar y solicitar confirmación (1: Sí / 2: No). Si elige "No", asignar `opcion = 0` y volver al paso 3.
 - 4.4.2. Si `datosProcesados == false` o se confirma la salida, cerrar el objeto Scanner.
5. **Fin del Programa.**

10. Desarrollo Pseint

Algoritmo AplicativoInteractivo2

// Constante academica para definir el limite minimo de aprobacion

Definir `NOTA_MINIMA_APROBACION` Como Real

`NOTA_MINIMA_APROBACION` <- 7.0

// Variables globales de control y datos

Definir `opcion` Como Entero

Definir `nombreEstudiante`, `lenguajeUtilizado` Como Cadena

Definir `datosProcesados` Como Logico

Definir `confirmarSalir` Como Entero

// Variables para almacenar los resultados del sistema

Definir `num1`, `num2`, `suma`, `resta`, `multiplicacion`, `division`, `promedio`, `notaMayor`, `notaMenor` Como Real

Definir `aprobados`, `reprobados` Como Entero

Definir `banderaOperaciones`, `banderaNotas` Como Logico

Definir `divisionCero` Como Logico

// Arreglo unidimensional para las calificaciones

Definir `notas` Como Real

Dimension notas[5]

// Inicializacion de estados

datosProcesados <- Falso

banderaOperaciones <- Falso

banderaNotas <- Falso

divisionCero <- Falso

Escribir "=====

Escribir " CONFIGURACION INICIAL DEL SISTEMA "

Escribir "=====

// Validacion de Nombre

Repetir

Escribir "Ingrese el nombre del estudiante: "

Leer nombreEstudiante

Si nombreEstudiante = "" Entonces

Escribir "Error: El campo del nombre no puede estar vacio."

FinSi

Hasta Que nombreEstudiante <> ""

// Validacion de Lenguaje

Repetir

Escribir "Ingrese el lenguaje de programacion (ej. Java): "

Leer lenguajeUtilizado

Si lenguajeUtilizado = "" Entonces

Escribir "Error: El campo del lenguaje no puede estar vacio."

FinSi

Hasta Que lenguajeUtilizado <> ""

Escribir "Sistema inicializado correctamente."

Escribir ""

// Menu interactivo continuo

Repetir

Escribir "=====

Escribir " APLICATIVO INTERACTIVO DE LOGICA "

Escribir "=====

Escribir " Estudiante activo: ", nombreEstudiante, " [", lenguajeUtilizado, "]"

Escribir "-----"

Escribir "1. Operaciones basicas"

Escribir "2. Registro de notas"

Escribir "3. Guardar resultados"

Escribir "4. Salir"

Escribir "=====

// Validacion de la opcion del menu

Repetir

Escribir "Seleccione una opcion (1 - 4): "

Leer opcion

Si opcion < 1 O opcion > 4 Entonces

Escribir "Error: Opcion fuera de rango."

FinSi

Hasta Que opcion >= 1 Y opcion <= 4

Segun opcion Hacer

1:

Escribir ""

Escribir "--- Módulo de Operaciones Basicas ---"

Escribir "Ingrese el primer numero: "

Leer num1

Escribir "Ingrese el segundo numero: "

Leer num2

suma <- num1 + num2

resta <- num1 - num2

multiplicacion <- num1 * num2

Si num2 <> 0 Entonces

division <- num1 / num2

divisionCero <- Falso

Sino

divisionCero <- Verdadero

FinSi

Escribir ""

Escribir "--- Resultados de las Operaciones ---"

Escribir "Suma: ", suma

Escribir "Resta: ", resta

Escribir "Multiplicacion: ", multiplicacion

Si divisionCero = Falso Entonces

Escribir "Division: ", division

Sino

Escribir "Division: Error: Division para cero no permitida."

FinSi

Escribir ""

banderaOperaciones <- Verdadero

datosProcesados <- Verdadero

2:

Definir i Como Entero

Definir sumaNotas Como Real

Escribir ""

Escribir "--- Módulo de Registro de 5 Notas ---"

sumaNotas <- 0

aprobados <- 0

reprobados <- 0

// Llenado seguro del vector con validacion de rango

Para i <- 1 Hasta 5 Con Paso 1 Hacer

Repetir

Escribir "Ingrese la nota de la asignatura ", i, " (0 - 10): "

Leer notas[i]

Si notas[i] < 0.0 O notas[i] > 10.0 Entonces

Escribir "Error: Calificacion fuera del rango academico (0 - 10)."

FinSi

Hasta Que notas[i] >= 0.0 Y notas[i] <= 10.0

FinPara

// Inicializacion de extremos con el primer valor real ingresado

notaMayor <- notas[1]

notaMenor <- notas[1]

// Procesamiento estadístico descriptivo del vector

Para i <- 1 Hasta 5 Con Paso 1 Hacer

 sumaNotas <- sumaNotas + notas[i]

 Si notas[i] > notaMayor Entonces

 notaMayor <- notas[i]

 FinSi

 Si notas[i] < notaMenor Entonces

 notaMenor <- notas[i]

 FinSi

 Si notas[i] >= NOTA_MINIMA_APROBACION Entonces

 aprobados <- aprobados + 1

 Sino

 reprobados <- reprobados + 1

 FinSi

FinPara

promedio <- sumaNotas / 5

Escribir ""

Escribir "--- Resultados del Procesamiento ---"

Escribir "Promedio General: ", promedio

Escribir "Nota Mas Alta: ", notaMayor

Escribir "Nota Mas Baja: ", notaMenor

Escribir "Asignaturas Aprobadas: ", aprobados

Escribir "Asignaturas Reprobadas: ", reprobados

Escribir ""

banderaNotas <- Verdadero

datosProcesados <- Verdadero

3:

Si datosProcesados Entonces

Escribir ""

Escribir "[SIMULACION DE PERSISTENCIA EN: resultados/resultados.txt]"

Escribir "=====

Escribir "Estudiante: ", nombreEstudiante

Escribir "Fecha y Hora: [Marcador de Tiempo Local]"

Escribir "Entorno / Lenguaje: ", lenguajeUtilizado

Escribir "-----"

Escribir "DETALLE DE RESULTADOS ACUMULADOS:"

Si banderaOperaciones Entonces

Escribir " > OPERACIONES MATEMATICAS:"

Escribir " - Numeros ingresados: ", num1, " y ", num2

Escribir " - Suma: ", suma

Escribir " - Resta: ", resta

Escribir " - Multiplicacion: ", multiplicacion

Si divisionCero = Falso Entonces

Escribir " - Division: ", division

Sino

Escribir " - Division: Error: Division para cero no permitida."

FinSi

FinSi

Si banderaNotas Entonces

Escribir " > PROCESAMIENTO DE NOTAS:"

Escribir " - Promedio General: ", promedio

Escribir " - Nota Mas Alta: ", notaMayor

Escribir " - Nota Mas Baja: ", notaMenor

Escribir " - Asignaturas Aprobadas: ", aprobados

Escribir " - Asignaturas Reprobadas: ", reprobados

FinSi

Escribir "=====."

Escribir "Exito: Datos guardados de manera local correctamente."

Escribir ""

// Limpieza de banderas de control tras el almacenamiento exitoso

datosProcesados <- Falso

banderaOperaciones <- Falso

banderaNotas <- Falso

Sino

Escribir "Aviso: No existen nuevos datos en memoria para guardar."

Escribir ""

FinSi

4:

Si datosProcesados Entonces

Escribir "Advertencia: Existen modificaciones pendientes sin guardar."

Repetir

Escribir "¿Desea salir sin guardar los resultados? (1: Si / 2: No): "

Leer confirmarSalir

Si confirmarSalir < 1 O confirmarSalir > 2 Entonces

Escribir "Error: Seleccione 1 o 2."

FinSi

Hasta Que confirmarSalir = 1 O confirmarSalir = 2

Si confirmarSalir = 2 Entonces

opcion <- 0 // Restablece la opcion para no romper el bucle principal

FinSi

FinSi

Si opcion = 4 Entonces

Escribir "Finalizando el hilo de ejecucion de la aplicacion. Hasta luego."

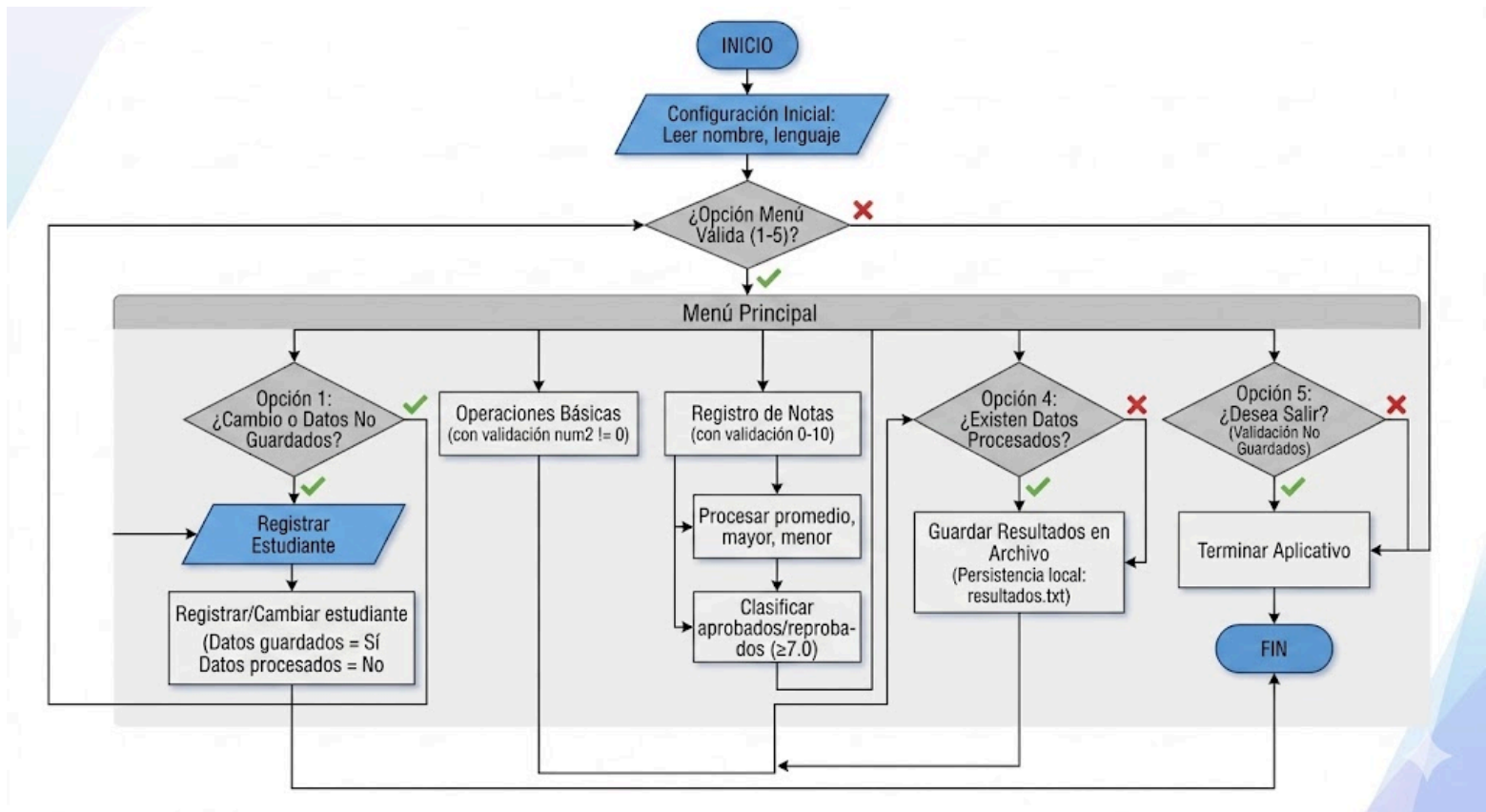
FinSi

FinSegun

Hasta Que opcion = 4

FinAlgoritmo

11. Diagrama de flujo



12. Actividades por Desarrollar

```
import java.io.File;
```

```
import java.io.FileWriter;
```

```
import java.io.IOException;
```

```
import java.io.PrintWriter;
```

```
import java.time.LocalDateTime;
```

```
import java.time.format.DateTimeFormatter;
```

```
import java.util.Scanner;
```

```
/**
```

* Aplicativo Interactivo de Logica y Algoritmos en Java.

* Asignatura: Algoritmos y Logica de Programacion.

* Universidad Tecnica de Ambato.

*/

```
public class AplicativoInteractivo2 {
```

```
    // Constante academica para definir el limite minimo de aprobacion
```

```
    private static final double NOTA_MINIMA_APROBACION = 7.0;
```

```
    // Scanner estatico para evitar redundancia de paso de parametros
```

```
    private static final Scanner scanner = new Scanner(System.in);
```

```
    public static void main(String[] args) {
```

```
        int opcion;
```

```
        // Variables para almacenar resultados intermedios y persistencia
```

```
        String nombreEstudiante = "";
```

```
        String lenguajeUtilizado = "";
```

```
        StringBuilder resultadosAcumulados = new StringBuilder();
```

```
        boolean datosProcesados = false;
```

```
        boolean estudianteConfigurado = false;
```

```
        // Parte 1: Menu interactivo continuo
```

```
        do {
```

```
            System.out.println("\n=====");
```

```
            System.out.println("  APLICATIVO INTERACTIVO DE LOGICA  ");
```

```
            System.out.println("=====");
```

```
            // Muestra informacion del estudiante activo en el menu si ya existe uno
```



```

if (estudianteConfigurado) {

    System.out.println(" Estudiante activo: " + nombreEstudiante + " [" + lenguajeUtilizado + "]");

    System.out.println("-----");

}

System.out.println("1. Registrar / Cambiar estudiante");

System.out.println("2. Operaciones basicas");

System.out.println("3. Registro de notas");

System.out.println("4. Guardar resultados en archivo");

System.out.println("5. Salir");

System.out.println("=====");

opcion = leerEnteroValido("Seleccione una opcion: ", 1, 5);

switch (opcion) {

    case 1:

        // Validacion de seguridad: Evita perder datos no guardados del estudiante anterior

        if (datosProcesados) {

            System.out.println("Aviso: Existen datos procesados del estudiante actual que no se han guardado.");

            System.out.print("¿Desea continuar y borrar estos datos? (1: Si / 2: No): ");

            int confirmar = leerEnteroValido("", 1, 2);

            if (confirmar == 2) {

                System.out.println("Operacion cancelada. Guarde los resultados primero.");

                break;

            }

        }

    }

}

```

```
System.out.println("\n=== CONFIGURACION DEL ESTUDIANTE ===");

nombreEstudiante = leerNombreValido();

lenguajeUtilizado = leerLenguajeValido();

resultadosAcumulados.setLength(0); // Limpia datos del estudiante anterior de manera segura

datosProcesados = false;

estudianteConfigurado = true;

System.out.println("Exito: Estudiante configurado correctamente.");

break;
```

case 2:

```
if (!estudianteConfigurado) {

    System.out.println("Aviso: Debe registrar un estudiante en la opcion 1 antes de operar.");

    break;

}

String resultadoOperaciones = ejecutarOperacionesBasicas();

resultadosAcumulados.append(resultadoOperaciones).append("\n");

datosProcesados = true;

break;
```

case 3:

```
if (!estudianteConfigurado) {

    System.out.println("Aviso: Debe registrar un estudiante en la opcion 1 antes de operar.");

    break;

}

String resultadoNotas = ejecutarRegistroNotas();

resultadosAcumulados.append(resultadoNotas).append("\n");

datosProcesados = true;
```

```
break;
```

case 4:

```
if (!estudianteConfigurado) {  
    System.out.println("Aviso: No hay ningun estudiante activo registrado.");  
} else if (datosProcesados) {  
    guardarResultados(nombreEstudiante, lenguajeUtilizado, resultadosAcumulados.toString().trim());  
    // Limpieza de estados tras persistencia exitosa para el siguiente calculo  
    resultadosAcumulados.setLength(0);  
    datosProcesados = false;  
} else {  
    System.out.println("Aviso: No existen nuevos datos procesados para guardar o ya fueron guardados.");  
}  
break;
```

case 5:

```
// Alerta si el usuario intenta salir con calculos pendientes sin almacenar  
if (datosProcesados) {  
    System.out.println("Aviso: Hay datos sin guardar.");  
    System.out.print("¿Esta seguro de que desea salir? (1: Si / 2: No): ");  
    int confirmarSalir = leerEnteroValido("", 1, 2);  
    if (confirmarSalir == 2) {  
        opcion = 0; // Cambia el estado de la opcion para no romper el bucle  
        break;  
    }  
}  
  
System.out.println("Saliendo del aplicativo. Hasta luego");
```

```
break;
```

```
default:
```

```
    System.out.println("Opcion no valida. Intente nuevamente.");
```

```
}
```

```
} while (opcion != 5);
```

```
scanner.close();
```

```
}
```

```
// Parte 2: Operaciones matemáticas
```

```
private static String ejecutarOperacionesBasicas() {
```

```
    System.out.println("\n--- Operaciones Basicas ---");
```

```
    double num1 = leerDoubleValido("Ingrese el primer numero: ");
```

```
    double num2 = leerDoubleValido("Ingrese el segundo numero: ");
```

```
    double suma = num1 + num2;
```

```
    double resta = num1 - num2;
```

```
    double multiplicacion = num1 * num2;
```

```
    String divisionStr;
```

```
    if (num2 != 0) {
```

```
        double division = num1 / num2;
```

```
        divisionStr = String.format("%.2f", division);
```

```
    } else {
```

```
        divisionStr = "Error: Division para cero no permitida.";
```

```

}

System.out.println("\n--- Resultados de las Operaciones ---");

System.out.printf("Suma: %.2f\n", suma);

System.out.printf("Resta: %.2f\n", resta);

System.out.printf("Multiplicacion: %.2f\n", multiplicacion);

System.out.println("Division: " + divisionStr);

return String.format(" > OPERACIONES MATEMATICAS:\n" +
    " - Numeros ingresados: %.2f y %.2f\n" +
    " - Suma: %.2f\n" +
    " - Resta: %.2f\n" +
    " - Multiplicacion: %.2f\n" +
    " - Division: %s\n",
    num1, num2, suma, resta, multiplicacion, divisionStr);
}

```

// Parte 3: Arreglos y procesamiento de notas

```

private static String ejecutarRegistroNotas() {

    System.out.println("\n--- Registro de 5 Notas ---");

    double[] notas = new double[5];

    double sumaNotas = 0;

    int aprobados = 0;

    int reprobados = 0;

    for (int i = 0; i < notas.length; i++) {

        notas[i] = leerNotaValida("Ingrese la nota de la asignatura " + (i + 1) + " (0 - 10): ", 0.0, 10.0);
    }
}

```

```
}
```

```
double notaMayor = notas[0];
```

```
double notaMenor = notas[0];
```

```
for (int i = 0; i < notas.length; i++) {
```

```
    double notaActual = notas[i];
```

```
    sumaNotas += notaActual;
```

```
    if (notaActual > notaMayor) {
```

```
        notaMayor = notaActual;
```

```
    }
```

```
    if (notaActual < notaMenor) {
```

```
        notaMenor = notaActual;
```

```
    }
```

```
// Uso adaptativo de la constante de evaluacion
```

```
if (notaActual >= NOTA_MINIMA_APROBACION) {
```

```
    aprobados++;
```

```
} else {
```

```
    reprobados++;
```

```
}
```

```
}
```

```
double promedio = sumaNotas / notas.length;
```

```
System.out.println("\n--- Resultados del Procesamiento ---");
```

```
System.out.printf("Promedio: %.2f\n", promedio);

System.out.printf("Nota Mayor: %.2f\n", notaMayor);

System.out.printf("Nota Menor: %.2f\n", notaMenor);

System.out.println("Cantidad de Asignaturas Aprobadas: " + aprobados);

System.out.println("Cantidad de Asignaturas Reprobadas: " + reprobados);
```

```
// CORREGIDO: Se cambia 'Estudiantes' por 'Asignaturas' para ser consistente con la logica
```

```
return String.format(" > PROCESAMIENTO DE NOTAS:\n" +

    " - Promedio General: %.2f\n" +

    " - Nota Mas Alta: %.2f\n" +

    " - Nota Mas Baja: %.2f\n" +

    " - Asignaturas Aprobadas: %d\n" +

    " - Asignaturas Reprobadas: %d\n",

    promedio, notaMayor, notaMenor, aprobados, reprobados);

}
```

```
// Parte 4: Persistencia en archivos
```

```
private static void guardarResultados(String nombre, String lenguaje, String resultados) {

    String carpetaDestino = "resultados";

    String rutaArchivo = carpetaDestino + "/resultados.txt";

    LocalDateTime fechaHoraActual = LocalDateTime.now();

    DateTimeFormatter formateador = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");

    String fechaFormateada = fechaHoraActual.format(formateador);

    File directorio = new File(carpetaDestino);

    if (!directorio.exists()) {
```

```

    directorio.mkdirs();

}

try (PrintWriter writer = new PrintWriter(new FileWriter(rutaArchivo, true))) {

    writer.println("=====");

    writer.println("Estudiante: " + nombre);

    writer.println("Fecha y Hora: " + fechaFormateada);

    writer.println("Entorno / Lenguaje: " + lenguaje);

    writer.println("-----");

    writer.println("DETALLE DE RESULTADOS:");

    writer.println(resultados);

    writer.println("=====\n");

    System.out.println("Exito: Datos persistidos correctamente en " + rutaArchivo + ".");

} catch (IOException e) {

    System.out.println("Error Critico de I/O: No se pudo escribir en el archivo: " + e.getMessage());

}

}

// =====

// METODOS DE VALIDACION OPTIMIZADOS (SIN TILDES EN LOS MENSAJES)

// =====

private static String leerNombreValido() {

    String entrada;

    while (true) {

        System.out.print("Ingrese el nombre del estudiante: ");

```



```

entrada = scanner.nextLine().trim();

if (entrada.isEmpty()) {

    System.out.println("Error: El campo del nombre no puede estar vacio.");

} else if (!entrada.matches("[a-zA-ZáéíóúÁÉÍÓÚñ\\s]+")) {

    System.out.println("Error: El nombre debe contener unicamente letras y espacios.");

} else {

    return entrada;

}

}

}

```

```

private static String leerLenguajeValido() {

    String entrada;

    while (true) {

        System.out.print("Ingrese el lenguaje de programacion utilizado actualmente (ej. Java / C++): ");

        entrada = scanner.nextLine().trim();

        if (entrada.isEmpty()) {

            System.out.println("Error: El campo del lenguaje no puede estar vacio.");

        } else {

            return entrada;

        }

    }

}

```

```

private static int leerEnteroValido(String mensaje, int min, int max) {

```

```
int valor;

while (true) {

    System.out.print(mensaje);

    String entrada = scanner.nextLine().trim();

    try {

        if (entrada.isEmpty()) {

            System.out.println("Error: El campo no puede estar vacio.");

            continue;

        }

        valor = Integer.parseInt(entrada);

        if (valor >= min && valor <= max) {

            return valor;

        } else {

            System.out.println("Error: Opcion fuera de rango (" + min + " - " + max + ").");

        }

    } catch (NumberFormatException e) {

        System.out.println("Error: Tipo de dato no valido. Ingrese un numero entero.");

    }

}
```

```
private static double leerDoubleValido(String mensaje) {

    double valor;

    while (true) {

        System.out.print(mensaje);

        String entrada = scanner.nextLine().trim().replace(',', '.');

        try {
```

```

    if (entrada.isEmpty()) {

        System.out.println("Error: El campo no puede estar vacio.");

        continue;

    }

    valor = Double.parseDouble(entrada);

    return valor;

} catch (NumberFormatException e) {

    System.out.println("Error: Formato numerico incorrecto. Ingrese un valor decimal o entero.");

}

}

}

```

```

private static double leerNotaValida(String mensaje, double min, double max) {

    double valor;

    while (true) {

        System.out.print(mensaje);

        String entrada = scanner.nextLine().trim().replace(',', '.');

        try {

            if (entrada.isEmpty()) {

                System.out.println("Error: El campo no puede estar vacio.");

                continue;

            }

            valor = Double.parseDouble(entrada);

            if (valor >= min && valor <= max) {

                return valor;

            } else {

                System.out.println("Error: Escala de evaluacion fuera de rango (" + min + " - " + max + ").");
            }
        }
    }
}

```

```
    }

    } catch (NumberFormatException e) {

        System.out.println("Error: Formato de nota incorrecto. Ingrese un decimal.");

    }

}

}

}
```

13. Resultados Obtenidos

13.1. Interfaz de Configuración y Menú Dinámico

El sistema fuerza de manera secuencial el ingreso de los datos iniciales, impidiendo cadenas vacías o formatos prohibidos. Una vez superado este control, el menú refleja el estado dinámico del estudiante en cada iteración del bucle, cumpliendo de esta forma con el primer objetivo específico.

13.2. Procesamiento Numérico y Control de Excepciones Aritméticas

Al ejecutar el módulo de operaciones con entradas de prueba (num1 = 15.50, num2 = 0.00), el sistema procesó la suma (15.50), resta (15.50) y multiplicación (0.00). Ante la división para cero, la validación condicional impidió la caída de la aplicación (runtime exception), capturando el evento mediante la cadena "Error: Division para cero no permitida."

13.3. Análisis Estadístico de Calificaciones (Uso de Arreglos)

En la evaluación del módulo de notas, se ingresaron de forma controlada los valores mostrados en la Tabla 1. El algoritmo calculó de manera exacta las métricas descriptivas, cumpliendo con el segundo objetivo específico al aislar la inicialización de extremos a partir del primer dato real ingresado (notas[0]).

Tabla 1. Resultados del Procesamiento del Vector de Calificaciones

Notas Ingresadas (Vector)	Promedio General	Nota Más Alta	Nota Más Baja	Asignaturas Aprobadas	Asignaturas Reprobadas
[8.5, 6.4, 9.2, 5.0, 7.8]	7.38	9.20	5.00	3 (≥ 7.0)	2 (< 7.0)

13.4. Estructura del Archivo de Persistencia Física

Al ejecutar la opción de guardado, la aplicación generó físicamente la estructura de directorios y persistió la información de manera acumulativa, cumpliendo con el tercer objetivo específico. A continuación se presenta el esquema estructural del texto plano generado en el archivo local.

=====

Estudiante: Castro Adrian

Fecha y Hora: 2026-05-13 09:02:31

Entorno / Lenguaje: Java

DETALLE DE RESULTADOS:

> OPERACIONES MATEMATICAS:

- Numeros ingresados: 12.50 y 4.20
- Suma: 16.70
- Resta: 8.30
- Multiplicacion: 52.50
- Division: 2.98

> PROCESAMIENTO DE NOTAS:

- Promedio General: 7.38
 - Nota Mas Alta: 9.20
 - Nota Mas Baja: 5.00
 - Asignaturas Aprobadas: 3
 - Asignaturas Reprobadas: 2
- =====

14. Habilidades Blandas Empleadas en la Práctica

- ---

 - [x] Pensamiento crítico (Diseño de validaciones lógicas y mitigación de errores).
- [x] Responsabilidad (Cumplimiento autónomo de los requerimientos y rúbricas).
- [x] Adaptabilidad (Migración y estructuración lógica de algoritmos basados en pseudocódigo hacia lenguaje Java).

15. Conclusiones

- ---

Se implementó con éxito un menú continuo guiado por la estructura do-while que actúa como el controlador central del aplicativo. Al asociar una variable booleana como bandera de estado (datosProcesados), se garantizó la integridad del flujo del programa, alertando eficazmente al usuario sobre la existencia de datos en memoria volátil antes de permitir la salida de la aplicación, satisfaciendo el primer objetivo específico.
- El uso de un arreglo unidimensional de tamaño fijo demostró ser la estructura de datos óptima para encapsular las calificaciones de forma indexada. El algoritmo diseñado resolvió de manera exacta el cálculo de la media aritmética y la clasificación de estados académicos; asimismo, al fijar los valores de comparación iniciales directamente desde la primera posición del vector (notas[0]), se neutralizó el riesgo de lecturas erróneas de extremos, dando cumplimiento al segundo objetivo específico.
- Se consolidó el almacenamiento persistente de datos mediante la integración de las clases File, FileWriter y PrintWriter. La configuración del canal de escritura en modo apéndice facilitó el registro histórico y ordenado de

los cálculos sin alterar los datos previos en el disco secundario, logrando indexar metadatos temporales de control de manera exitosa y cumpliendo con el tercer objetivo específico.

16. Recomendaciones

- Incorporar una opción dentro del menú principal que permita al usuario restablecer o reconfigurar los metadatos iniciales del perfil del estudiante (`nombreEstudiante` y `lenguajeUtilizado`) en tiempo de ejecución, evitando la necesidad de reiniciar completamente la aplicación para cambiar de operador, en concordancia con el primer objetivo específico.
- Modularizar el análisis estadístico del vector de notas mediante la implementación de funciones dedicadas e independientes (por ejemplo, métodos separados para `calcularPromedio`, `obtenerNotaMaxima` y `obtenerNotaMinima`), lo cual elevará la cohesión del software y facilitará futuras pruebas unitarias sobre el arreglo de datos, apoyando lo planteado en el segundo objetivo específico.
- Modificar el mecanismo de persistencia para que el sistema realice una validación previa de lectura sobre el archivo `resultados.txt` antes de escribir, o implementar un formato de serialización estructurado como JSON o CSV, facilitando la futura migración de los datos históricos hacia bases de datos relacionales o herramientas analíticas externas, conforme al tercer objetivo específico.

17. Referencias Bibliográficas

-
- [1] H. M. Deitel y P. J. Deitel, *Cómo Programar en Java*, 10.^a ed. México: Pearson Educación, 2016.
 - [2] J. Bloch, *Effective Java*, 3.^a ed. Boston, MA: Addison-Wesley, 2018.
 - [3] I. Estudiante, "PRUEBA_PRACTICA," GitHub, 2026. [En línea]. Disponible en:
https://github.com/isma1402uta-boop/PRUEBA_PRACTICA

17. Anexos

```

run:
=== CONFIGURACION INICIAL DEL APLICATIVO ===
Ingrese el nombre del estudiante: Adrian
Ingrese el lenguaje de programacion utilizado actualmente por el estudiante (ej. Java / C++): C++

=====
      APLICATIVO INTERACTIVO DE LOGICA
=====

1. Operaciones basicas
2. Registro de notas
3. Guardar resultados
4. Salir
Seleccione una opcion: 2

--- Registro de 5 Notas ---
Ingrese la nota de la asignatura 1 (0 - 10): 10
Ingrese la nota de la asignatura 2 (0 - 10): 10
Ingrese la nota de la asignatura 3 (0 - 10): 10
Ingrese la nota de la asignatura 4 (0 - 10): 10
Ingrese la nota de la asignatura 5 (0 - 10): 10

--- Resultados del Procesamiento ---
Promedio: 10,00
Nota Mayor: 10,00
Nota Menor: 10,00
Cantidad de Asignaturas Aprobadas: 5
Cantidad de Asignaturas Reprobadas: 0

```

Interfaz de usuario y ejecución del sistema de gestión académica en entorno de desarrollo integrado [1]. Fuente: Elaboración propia.

```

=====
      APLICATIVO INTERACTIVO DE LOGICA
=====

1. Operaciones basicas
2. Registro de notas
3. Guardar resultados
4. Salir
Seleccione una opcion: 2

--- Registro de 5 Notas ---
Ingrese la nota de la asignatura 1 (0 - 10): 3
Ingrese la nota de la asignatura 2 (0 - 10): 4
Ingrese la nota de la asignatura 3 (0 - 10): 5
Ingrese la nota de la asignatura 4 (0 - 10): 6
Ingrese la nota de la asignatura 5 (0 - 10): 7

--- Resultados del Procesamiento ---
Promedio: 5,00
Nota Mayor: 7,00
Nota Menor: 3,00
Cantidad de Asignaturas Aprobadas: 1
Cantidad de Asignaturas Reprobadas: 4

```

Interfaz de usuario y ejecución del sistema de gestión académica en entorno de desarrollo integrado mediante validaciones de entradas [1]. Fuente: Elaboración propia.

```
=====
      APLICATIVO INTERACTIVO DE LOGICA
=====
1. Operaciones basicas
2. Registro de notas
3. Guardar resultados
4. Salir
Seleccione una opcion: 1

--- Operaciones Basicas ---
Ingrese el primer numero: 5
Ingrese el segundo numero: 6

--- Resultados de las Operaciones ---
Suma: 11,00
Resta: -1,00
Multiplicacion: 30,00
Division: 0,83
```

Interfaz de usuario y ejecución del sistema de gestión académica en entorno de desarrollo integrado mediante validación de datos de entrada múltiple [1]. Fuente: Elaboración propia.